

Conversation memory

LESSON 14 · MODULE 3 — KNOWLEDGE & MEMORY

What you'll learn

- How to wrap `agent.run()` in a small **chat helper** so you can hold a real multi-turn conversation
- How a rolling `memory=20` window behaves across several exchanges
- The production pattern for keeping different users' conversations separate: **one `session_id` per user**

Key concepts

A chat helper turns one-off calls into a conversation. Instead of scattering `agent.run()` calls, write a small function that prints both sides of the exchange and threads the *same* `session_id` through every call. `memory=20` gives a rolling window of the last twenty messages — enough for a real back-and-forth — and the helper just takes a message and a session ID and lets the agent thread them together.

Memory does the work across turns. Turn one asks for an onboarding checklist. Turn two adds "remote, from London" and the agent adjusts — UK shipping, remote-first, MDM, time-zone and support coverage. Turn three just says "do all of this" and the agent threads a full timeline against everything from the previous two turns. No re-stating context; the window carries it.

One agent, one `session_id` per user, is the multi-tenant pattern. The same agent serves many users — you keep them separate by giving each user their own `session_id`. Ask the same identifying question in Sarah's session and Marcus's session and each answer stays scoped to that user; neither leaks into the other. This is the shape for any multi-user or multi-tenant chatbot: shared agent, per-user session.

TAKEAWAY

A chat helper turns one-off calls into a conversation.

TAKEAWAY

Memory does the work across turns.

Code shown in this lesson

PYTHON

```
# A rolling window big enough for a real exchange.
agent = studio.create_agent(
    name="onboarding-helper", provider="gpt-4o",
    role="employee onboarding assistant",
    goal="build and adjust onboarding checklists across a conversation",
    memory=20, # last 20 messages
)

# Chat helper: threads one session_id through every turn and prints both sides.
def chat(message, session_id):
    print(f"you> {message}")
    resp = agent.run(message, session_id=session_id)
    print(f"agent> {resp.response}\n")
    return resp
```

PYTHON

```
# Three exchanges in one session — memory threads the context.
sid = "user-alex"
chat("Draft an onboarding checklist for a new hire.", sid)
chat("They're remote, based in London.", sid) # agent adjusts: UK shipping, MDM, time zones
chat("Great, now give me the full timeline for all of this.", sid) # threads turns 1 + 2
```

PYTHON

```
# Multi-tenant isolation: same agent, one session_id per user.
chat("Remind me who I am and what my team does.", "user-sarah") # knows Sarah / design system
chat("Remind me who I am and what my team does.", "user-marcus") # knows Marcus / platform
# Neither session leaks into the other.
```

TRY THIS

- 1** Write the `chat()` helper and run a 3-turn conversation where each turn depends on the last. Drop `memory` to `2` and rerun the same script — watch the agent lose the early context once it falls out of the window.
- 2** Run the same identifying question under two different `session_id`s in the same process. Confirm zero cross-talk — that's your multi-tenant isolation proof.
- 3** Reuse one `session_id` across what should be two *different* users and watch their context bleed together — the bug the per-user-session pattern prevents.